

Driblab Capstone — Executive Summary

Steps 1 & 2: Data Analysis and Alignment Foundation

Match used for validation: 678949 — Manchester United vs Sunderland (ENG I 2025)

Contexto del proyecto

El objetivo del capstone es construir un sistema que detecte y clasifique eventos de fútbol (pases, tackles, tiros, intercepciones...) a partir de **tracking data**: coordenadas (x, y) de jugadores y balón, muestreadas 10 veces por segundo. Para una parte del dataset tenemos también **eventing data** (etiquetas ground truth de lo que ocurrió, cuándo y dónde), que usaremos para entrenar y evaluar.

El Kickoff Guide de Driblab impone un orden muy claro: **entender los datos antes de tocar cualquier modelo**. La razón es que la mayoría de los "bugs de modelo" en este tipo de proyectos son en realidad bugs de datos: coordenadas en espacios distintos, relojes desincronizados, posiciones imputadas confundidas con posiciones reales. Arreglar eso después de haber entrenado cuesta semanas.

Step 1 — Exploratory Data Analysis

Qué se hizo

Se analizaron sistemáticamente los tres tipos de fichero que forman el dataset:

- `dim_event_type.csv` — diccionario de tipos de eventos
- `*_events.json` — etiquetas de eventos por partido
- `*_tracking_data.jsonl` — posiciones de jugadores y balón frame a frame

Y se verificó la consistencia entre los dos últimos (cross-file consistency).

1a. Vocabulario de eventos (`dim_event_type.csv`)

Qué encontramos: El CSV define 84 tipos de eventos. Pero en los datos reales de un partido solo aparecen 23. Los 61 restantes no ocurrieron o son tipos que Driblab reserva para situaciones muy específicas (events ID ≥ 100 son subtipos derivados como `LONG BALL`, `CROSS`, `KEY PASS` — anotaciones adicionales sobre un PASS base, no eventos independientes).

Por qué importa: Si intentas detectar los 84 tipos tendrás clases con cero ejemplos. El modelo no puede aprender de lo que no existe en los datos. Hay que quedarse solo con los tipos que realmente ocurren.

Decisión: Trabajar con los **23 tipos presentes**, y de esos, filtrar los administrativos (START, END, SUBSTITUTION...) que no son detectables desde tracking.

1b. Distribución de clases

Qué encontramos:

Evento	Frecuencia	%
PASS	1.217	69.5%
BALL TOUCH	148	8.4%
TACKLE	66	3.8%
TAKEON	53	3.0%
FOUL	45	2.6%
BALL RECOVERY	41	2.3%
INTERCEPTION	32	1.8%
AERIAL	30	1.7%
... shots, goals	<20	<1%

Implicación crítica: PASS domina al 69.5%. Un clasificador que predice "PASS" para todo obtiene un 70% de accuracy sin aprender nada. Por eso **la métrica correcta es per-class F1, nunca accuracy global**. Esta es la trampa más común en proyectos de este tipo.

Decisión para el modelo: Los eventos worth detecting son los 7-8 con suficientes ejemplos para aprender: PASS, TACKLE, TAKEON, FOUL, BALL RECOVERY, INTERCEPTION, AERIAL. Los shots (SAVED SHOT, MISSED SHOT, GOAL) son demasiado raros para un clasificador multiclase fiable — se pueden agrupar como clase "SHOT".

1c. Sistema de coordenadas de los eventos

Qué encontramos: Las coordenadas (x) e (y) de los eventos están en espacio **normalizado 0-100**, y están normalizadas por **dirección de ataque**: para cualquier equipo, en cualquier periodo, el eje de ataque siempre apunta hacia $x=100$. Esto se verificó comprobando que los tiros de ambos equipos se agrupan cerca de $x=80-100$, independientemente del periodo.

Por qué se eligió esta normalización: Facilita el análisis (un pase progresivo siempre tiene $x_{end} > x_{start}$) pero introduce una asimetría importante respecto al tracking data, como veremos.

1d. Codificación del tiempo en eventos

Qué encontramos: El reloj usa $(period_id, min, sec, milisec)$. El campo (min) **no resetea a 0 en el descanso** — el periodo 2 empieza en $(min=45)$. Esto parece contraintuitivo pero es el reloj de fútbol estándar.

Por qué importa: Si asumes que min resetea y construyes una función de sincronización con esa premisa, cada evento del segundo tiempo quedará mal alineado con el tracking. Se documentó explícitamente para evitar este bug.

1e. Estructura del tracking data

Qué encontramos — fichero JSONL:

La primera línea es un header con metadata del partido y un diccionario de jugadores.
Las líneas siguientes son frames con esta estructura:

```
{
  "frame": 26,
  "Videotimestamp": 3.52,      ← segundos de vídeo transcurridos
  "match_clock": [0, 3],      ← [minutos, segundos] desde el kick-off
  "period": 1,
  "ball": [43.59, 42.51, 0.02], ← [x, y, z] en metros
  "cam": [[x0,y0], [x1,y1], [x2,y2], [x3,y3]],
  "data": {
    "1946": [{"id": 39298, "vis": false, "x": 74.85, "y": 31.92}, ...],
    "1925": [{"id": 1221508, "vis": false, "x": 51.59, "y": 16.87}, ...]
  }
}
```

Hallazgos importantes:

- **Coordenadas en metros físicos** (0-105 m × 0-68 m), no en espacio normalizado. Sistema de coordenadas distinto al de los eventos.
- **10 FPS confirmados** (el header declara `FPS: 10`) y el timestamp lo verifica).
- **22 jugadores por frame** en el 99.1% de frames activos (11 por equipo).
- El campo `vis` indica si la posición fue detectada directamente por visión artificial (True) o estimada por el sistema de imputación AI (False). El 64.3% de posiciones son imputadas.
- El campo `cam` es el polígono de la cámara de broadcast en coordenadas de pitch. Cuando es `None` significa que la cámara no está enfocando el campo (replay, plano del público, zoom). En esos frames las posiciones no están ancladas geométricamente.

1f. El flag `vis` — por qué importa

Qué significa `vis=False`: La posición fue **predicha por un modelo de IA** entrenado en datos de GPS, no detectada por el tracker de vídeo. Ocurre cuando el jugador está fuera del plano de cámara, ocluido por otro jugador, o el tracker lo pierde.

Por qué se usa IA en lugar de dejar NaN: Porque la mayoría de los modelos de detección de eventos necesitan las posiciones de todos los jugadores, no solo los visibles. La imputación AI hace el dato usable, pero a costa de precisión posicional.

Implicación para el modelado: Para features que requieren precisión espacial alta (distancia al defensor más cercano, ángulo de pase) conviene filtrar o ponderar los jugadores con `vis=False`. Para features más globales (centroide del equipo, número de jugadores en zona) el impacto es menor.

1g. Completitud del balón

Resultado (corregido en Step 2):

Métrica	Valor
Completitud global	48.8%
Completitud en juego real (cam presente)	80.7%

El 48.8% inicial era engañoso porque incluía frames sin cámara (replays, planos de estadio). El número relevante es 80.7% durante juego activo. El balón falta principalmente durante balones parados, corner setup, y oclusiones breves.

Implicación: El balón es la señal más importante del sistema de detección de eventos, pero hay que tener una estrategia para los gaps. La solución estándar es interpolación lineal para gaps cortos ($< 0.5s = 5$ frames) y descarte para gaps largos.

1h. Dataset disponible

Resultado del inventario:

Tipo	Cantidad
Partidos con eventos + tracking	33
Partidos solo con eventos	1
Total eventos etiquetados	~52.000

33 partidos con ambos assets es suficiente para construir un modelo, pero no es un dataset grande. Esto favorece métodos data-efficient (gradient boosting sobre features manuales) sobre deep learning.

Conclusiones del Step 1

1. Los datos son utilizables pero requieren transformación no trivial antes de modelar.
2. Hay dos sistemas de coordenadas distintos que deben alinearse.
3. Los relojes de eventos y tracking tienen distinta lógica y necesitan sincronización.
4. La clase PASS domina masivamente — la métrica de evaluación debe ser F1 por clase.
5. Solo 7-8 tipos de eventos tienen suficientes ejemplos para ser detectados.
6. El 64% de posiciones son imputadas — hay que tratarlas con cautela en features de precisión.

Step 2 — Alignment Foundation

Qué se hizo

Se construyeron las funciones de transformación necesarias para poder comparar coordenadas de eventos con coordenadas de tracking, y se validó visualmente que el resultado es correcto.

2a. Funciones de conversión de coordenadas

El problema: Los eventos están en espacio 0-100 normalizado. El tracking está en metros físicos $0-105 \times 0-68$. Son incomparables directamente.

La solución:

```
python
```

```
def event_coords_to_metres(x, y):
    return x / 100.0 * 105, y / 100.0 * 68

def tracking_coords_to_norm(x, y):
    return x / 105.0 * 100, y / 68.0 * 100
```

Por qué esta fórmula y no otra: La relación es lineal y el origen es el mismo (esquina inferior izquierda en ambos sistemas). Solo hay que escalar. Se eligió convertir eventos a metros (no al revés) porque los features del modelo se computarán principalmente en espacio de tracking, que es donde están las posiciones de jugadores.

Validación: `event_coords_to_metres(50, 50)` → `(52.5m, 34.0m)` = centro exacto del campo. Todas las assertions pasaron.

2b. Dirección de ataque en el tracking

El problema: Los eventos están normalizados por dirección de ataque (high x = gol del equipo que ataca). El tracking **no está normalizado** — los equipos cambian de lado en el descanso. Esto significa que en el periodo 2 las coordenadas x del tracking apuntan al lado contrario.

Cómo se detectó la dirección: Se tomó el primer frame del periodo 1 con jugadores y se calculó el centroide de cada equipo:

```
Manchester United: mean_x = 66.7m → en su propio campo defensivo (lado derecho)
Sunderland:      mean_x = 47.3m → en su propio campo defensivo (lado izquierdo)
```

Esto significa que en el P1, Man Utd ataca hacia la izquierda (x decreciente) y Sunderland hacia la derecha (x creciente). En P2 se invierten.

Por qué detectar desde datos y no hardcodear: Porque la dirección de ataque varía por partido y no está documentada en el header del fichero. El único método robusto es inferirla de las posiciones iniciales.

La función:

```
python

def normalize_tracking_direction(x, period, home_attacks_right_p1):
    needs_flip = home_attacks_right_p1 if period == 1 else not home_attacks_r
    return 105.0 - x if needs_flip else x
```

Todas las assertions pasaron.

2c. Sincronización evento-frame (nuevo hallazgo crítico)

El problema encontrado en Step 2 (no estaba en el EDA): El reloj del tracking (`match_clock`) **no resetea en el descanso**. Continúa desde donde terminó el P1. Esto es diferente a los eventos, donde el P2 empieza en `min=45`.

Medición concreta:

```
P1 último frame:  clock = [52, 38]
P2 primer frame:  clock = [52, 39]  ← continúa
P2 primer evento: min=45, sec=8
```

Offset P2 = 52:39 (tracking) - 45:00 (evento) = 7 min 39s = 459 segundos

La función de sincronización convierte cualquier evento a su frame más cercano en el tracking:

```
python

def find_nearest_frame(frames, period, min, sec):
    event_s = min * 60 + sec
    if period == 2:
        event_s += P2_OFFSET_S  # 459s para este partido
    # busca el frame con match_clock más cercano a event_s
```

Por qué esto es crítico: Sin este offset, cada evento del P2 quedaría mapeado al frame equivocado — 7 minutos y medio antes en el tracking. Las features de posición de jugadores estarían completamente desalineadas con la etiqueta del evento. El modelo aprendería ruido.

Drift real entre relojes: 0 segundos en el kick-off. No hay drift acumulativo — los relojes son consistentes una vez aplicado el offset del P2.

2d. Validación visual de la alineación

Cómo se validó: Se tomaron 10 eventos de PASS del periodo 1, se localizó el frame de tracking correspondiente, y se pintó la posición del evento (convertida a metros) sobre el mapa de posiciones de los jugadores de ese frame. Si la alineación es correcta, el punto rojo del evento debería caer encima o muy cerca del jugador que hizo el pase.

Resultado: 6/10 eventos dentro de 5 metros de un jugador.

Por qué 6/10 y no 10/10: Hay tres posibles causas, que el Step 2 deja pendiente de diagnóstico:

1. **Latencia del evento:** Opta registra el evento en el momento de la acción, pero puede haber 0.1–0.3s de diferencia con el frame exacto. En 0.2 segundos un jugador a 7 m/s recorre 1.4 metros — acumulable.
2. **vis=False players:** Si el jugador que hizo el pase tiene `vis=False` en ese frame, su posición es estimada y puede diferir varios metros de la real.
3. **Frame de búsqueda:** El frame exacto puede ser el anterior o el siguiente al más cercano por reloj.

Lo que hay que hacer: Diagnosticar los 4 eventos fallidos comprobando si el jugador responsable está en el frame y a qué distancia. Pendiente para inicio del Step 3.

Conclusiones del Step 2

1. Las funciones de conversión de coordenadas están construidas y verificadas.
2. La dirección de ataque se infiere correctamente de los datos — no hay que hardcodearla.
3. El offset del P2 (459s para el partido 678949) es el hallazgo más importante del step: sin él, el P2 entero queda desincronizado.

4. La alineación visual es correcta al 60% con tolerancia de 5m, lo que es suficiente para continuar pero hay que investigar los casos fallidos antes de construir features.
 5. La completitud real del balón es 80.7% durante juego activo — viable para usar como señal principal.
-

Estado actual y próximos pasos

Completado

- ☑ Step 1: Análisis exploratorio completo de los tres tipos de fichero
- ☑ Step 2: Funciones de alineación de coordenadas, dirección y reloj

Pendiente inmediato (antes de modelar)

- ☐ Diagnosticar el 40% de eventos con alineación > 5m
- ☐ Verificar que el P2 offset varía por partido o es constante
- ☐ Construir el filtro de "live play" (cam != None) para excluir frames no fiables

Step 3 (siguiente)

- Possession backbone: asignar el balón al jugador más cercano y detectar cambios de posesión
- Construcción de features: velocidad del balón, distancias, quién tiene la posesión

Step 4 (después)

- Detector rule-based: pase = balón va a compañero, intercepción/tackle = rival lo recupera, tiro = balón va hacia portería rápido
- Evaluación: precision/recall/F1 por clase en un partido que no se usó para ajustar las reglas

Step 5 (posterior)

- Clasificador ML (gradient boosting sobre features manuales) para los eventos ambiguos
 - Nunca optimizar accuracy — siempre F1 por clase con class weighting
-

Decisiones de diseño — preguntas frecuentes

¿Por qué gradient boosting y no una red neuronal? Solo tenemos ~52.000 eventos etiquetados distribuidos en 33 partidos. Las redes neuronales profundas necesitan órdenes de magnitud más datos para generalizar. Gradient boosting sobre features manuales (velocidad del balón, distancias, posesión) es más data-efficient, más interpretable, y típicamente outperforma a las redes en tabular data de tamaño medio.

¿Por qué no usar todos los 84 tipos de eventos? Porque 61 de ellos no ocurren en los datos, y los que sí ocurren tienen distribuciones extremadamente desiguales. Un modelo multiclase con clases sin ejemplos es no entrenable. Nos quedamos con 7-8 clases con suficientes ejemplos.

¿Por qué no interpolamos las posiciones con `vis=False`? Las posiciones `vis=False` ya son el resultado de la imputación AI de Driblab — no hay nada más que interpolar. Lo que sí podemos hacer es usarlas con menor peso en features de precisión, o crear una feature binaria "jugador visible en este frame" para que el modelo aprenda a ignorar las posiciones poco fiables.

¿Por qué el P2 offset no es siempre el mismo? El offset depende de cuándo terminó realmente el primer tiempo (incluyendo tiempo de descuento). En el partido 678949 fue 7 min 39s, pero en otro partido puede ser 6 o 9 minutos. Hay que calcularlo dinámicamente para cada partido a partir del primer frame del P2 en el tracking.

¿Por qué usar `match_clock` para sincronizar y no `Videotimestamp`? `Videotimestamp` cuenta segundos de vídeo desde el inicio de la grabación — incluye tiempo de calentamiento, ceremonias, descanso, y cualquier corte de la emisión. No tiene relación directa con el reloj de partido. `match_clock` sí cuenta tiempo de juego desde el kick-off, que es la misma referencia que usan los eventos.

Documento generado para la presentación interna — Driblab Capstone 2026